string

In [2]:
```
s = "HELLO WORLD"
```

In [4]:
```
print(s)
print(type(s))
```

```
HELLO WORLD
<class 'str'>
```

string indexing

python supports both : positive and negative index

In [10]:
```
print(s[0])
print(s[-len(s)])
```

```
H
H
```

In [12]:
```
print(s[len(s)-1])
print(s[-1])
```

```
D
D
```

string is immutable object

In [15]:
```
s[0] = "M"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[15], line 1
----> 1 s[0] = "M"

TypeError: 'str' object does not support item assignment
```

string slicing

In [20]:
```
# s[start:stop:step]
# default start = 0
# default stop = len(s)
# default step = 1
```

In [29]:
```
# First 5 characters
print(s[0:5:1])
print(s[:5])
```

```
HELLO
HELLO
```

In [31]:
```
# Characters from index 3 to 9
print(s[3:10])
```

```
LO WORL
```

In [35]:
```
# Entire string using slicing
print(s[0:len(s):1])
print(s[::])
```

```
HELLO WORLD
HELLO WORLD
```

In [39]:
```
# Characters from index 5 to end
print(s[5:len(s):1])
print(s[5:])
```

```
 WORLD
 WORLD
```

In [41]:
```
# Characters from beginning to index 4
print(s[0:5:1])
print(s[:5])
```

```
HELLO
HELLO
```

In [43]:
```
# Extract "Hello"
print(s[:5])
```

```
HELLO
```

In [45]:
```
# Extract "World"
print(s[6:])
```

```
WORLD
```

In [47]:
```
# Last 5 characters
print(s[-5:0:1]) #returns empty string
```

In [49]:
```
# Last 5 characters
print(s[-5::])
print(s[-5:])
```

```
WORLD
WORLD
```

In [51]:
```
# First 4 characters
print(s[:4])
```

```
HELL
```

In [53]:
```
s1 = "PYTHON PROGRAMMING"
```

In [61]:
```
# Extract PYTHON
print(s1[:6])
print(s1[-18:-12])
```

```
PYTHON
PYTHON
```

In [65]:
```
# Extract PROGRAM
print(s1[7:14])
print(s1[-11:-4])
```

```
PROGRAM
PROGRAM
```

In [67]:
```
# Extract GRAM
print(s1[10:14])
print(s1[-8:-4])
```

```
GRAM
GRAM
```

In [69]:
```
# Extract THON PRO
print(s1[2:10])
print(s1[-16:-8])
```

```
THON PRO
THON PRO
```

In [75]:
```
# Extract GNIMMARGORP (reverse of “PROGRAMMING”)
print(s1[:6:-1])
print(s1[:-12:-1])
```

```
GNIMMARGORP
GNIMMARGORP
```

In [81]:
```
# Reverse only "PYTHON" portion
print(s1[5::-1])
print(s1[-13::-1])
```

```
NOHTYP
NOHTYP
```

In [83]:
```
# Last 5 characters of the string
print(s1[-5:])
```

```
MMING
```

In [87]:
```
# First 3 characters in reverse order
print(s1[2::-1])
```

```
TYP
```

In [93]:
```
# Reverse the whole string
print(s1[len(s1)-1::-1])
print(s1[::-1])
```

```
GNIMMARGORP NOHTYP
GNIMMARGORP NOHTYP
```

Palindrom check

In [96]:
```
name = "nayan"
if name == name[::-1]:
    print("Palindrom")
else:
    print("not palindrom")
```

```
Palindrom
```

iterating a string

In [99]:
```
s = "hello"
```

In [101…

```
for i in s:
    print(i)
```

```
h
e
l
l
o
```

In [103…

```
for i in range(len(s)):
    print(i, s[i])
```

```
0 h
1 e
2 l
3 l
4 o
```

string concatenation

In [106…

```
s1 = "wel"
s2 = "come"
s3 = s1 + s2
print(s3)
```

```
welcome
```

In [108…

```
s1 = "wel"
s2 = "come"
s3 = s1 + " " + s2 #if extra space is neede
print(s3)
```

```
wel come
```

string repetition

In [111…

```
s4 = s * 5
print(s4)
```

```
hellohellohellohellohello
```

In [113…

```
s4 = s * 2.5
print(s4)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[113], line 1
----> 1 s4 = s * 2.5
      2 print(s4)

TypeError: can't multiply sequence by non-int of type 'float'
```

Problem : Display Cafe Menu

In [ ]:

```
#        MENU
#Tea...............10
# Coffee............50
```

```
# Bournvita.........70
# Freshlime.........30
```

In [116…

```
item1, price1 = "Tea", "10"
item2, price2 = "Coffee", "50"
item3, price3 = "Bournvita", "70"
item4, price4 = "Freshlime", "30"
print(f"{"MENU" :^20}")
print(item1 + ("." * (20-len(item1)-len(price1)) + price1))
print(item2 + ("." * (20-len(item2)-len(price2)) + price2))
print(item3 + ("." * (20-len(item3)-len(price3)) + price3))
print(item4 + ("." * (20-len(item4)-len(price4)) + price4))
```

```
        MENU
Tea...............10
Coffee............50
Bournvita.........70
Freshlime.........30
```

Problem : Display Credit Card Number

In [119…

```
# input: 1111 2222 3333 4444
# output:XXXX XXXX XXXX 4444
```

In [121…

```
card_no = input("Enter your creadit card number:")
print((("X"*4)+" ")*3 + card_no[-4:])
```

```
XXXX XXXX XXXX 4444
```

find(sub,start,stop) , rfind(sub,start,stop), index(sub,start,stop), rindex(sub,start,stop), count(sub)

In [124…

```
s = "Hello how are you"
```

In [126…

```
s.find("o")
```

Out[126…

```
4
```

In [128…

```
s.find("o",5)
```

Out[128…

```
7
```

In [130…

```
s.find("o",0,4)
```

Out[130…

```
-1
```

In [132…

```
s.rfind("o")
```

Out[132…

```
15
```

In [134…

```
s.index("o")
```

Out[134…

```
4
```

In [136…

```
s.index("z")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[136], line 1
----> 1 s.index("z")

ValueError: substring not found
```

In [138…
```
s.find("z")
```

Out[138…
```
-1
```

In [140…
```
s.rindex("o")
```

Out[140…
```
15
```

In [142…
```
s.count("o")
```

Out[142…
```
3
```

In [144…
```
s.count("z")
```

Out[144…
```
0
```

ljust(width,char), rjust(width,char), center(width,char), zfill(width)

In [147…
```
s = "Hello"
```

In [149…
```
len(s)
```

Out[149…
```
5
```

In [151…
```
s.ljust(10)
```

Out[151…
```
'Hello     '
```

In [155…
```
s.ljust(10,"_")
```

Out[155…
```
'Hello_____'
```

In [157…
```
s.ljust(10,"ab")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[157], line 1
----> 1 s.ljust(10,"ab")

TypeError: The fill character must be exactly one character long
```

In [159…
```
s.rjust(10)
```

Out[159…
```
'     Hello'
```

In [161…
```
s.rjust(10,".")
```

Out[161… '.....Hello'

In [165…
```
s.center(10,"*")
```

Out[165… '**Hello***'

In [167…
```
s = "123"
```

In [169…
```
s.zfill(8)
```

Out[169… '00000123'

lstrip(chars), rstrip(chars), strip(chars), replace(old,new,count)

In [172…
```
s = "      Hello     "
```

In [174…
```
s.lstrip()
```

Out[174… 'Hello   '

In [176…
```
s.rstrip()
```

Out[176… '   Hello'

In [178…
```
s.strip()
```

Out[178… 'Hello'

In [182…
```
s = "_____Hello"
```

In [184…
```
s.lstrip()
```

Out[184… '_____Hello'

In [186…
```
s.lstrip("_")
```

Out[186… 'Hello'

In [188…
```
s = "|_%@ _Hello|_%@ _"
```

In [190…
```
s.strip("|_@ %")
```

Out[190… 'Hello'

In [192…
```
s = "|_%@ _Hello|_%@ _"
s.strip("_@ %")
```

Out[192… '|_%@ _Hello|'

In [194…
```
s = "a-b-c-d-e"
s.replace("-","/")
```

Out[194… 'a/b/c/d/e'

In [196…

```
s = "a-b-c-d-e"
s.replace("-"," ")
```

Out[196…

```
'a b c d e'
```

In [198…

```
s = "a-b-c-d-e"
s.replace("-","")
```

Out[198…

```
'abcde'
```

split(sub, maxsplit), rsplit(sub, maxsplit) , join(iterable)

In [203…

```
s = "hi hello how are you?"
li = s.split()
print(li)
print(type(li))
```

```
['hi', 'hello', 'how', 'are', 'you?']
<class 'list'>
```

In [207…

```
s = "hi, hello, how, are, you"
li = s.split(",")
print(li)
print(type(li))
```

```
['hi', ' hello', ' how', ' are', ' you']
<class 'list'>
```

In [209…

```
s = "hi, hello, how, are, you"
li = s.split(",",2)
print(li)
print(type(li))
```

```
['hi', ' hello', ' how, are, you']
<class 'list'>
```

In [211…

```
s = "hi, hello, how, are, you"
li = s.rsplit(",",1)
print(li)
print(type(li))
```

```
['hi, hello, how, are', ' you']
<class 'list'>
```

In [213…

```
li = ["hi","hello","how","are","you"]
s = " ".join(li)
print(s)
```

```
hi hello how are you
```

In [215…

```
print(".".join("abcd"))
```

```
a.b.c.d
```

In [ ]:

```
# s.startswith(prefix)
# s.endswith(suffix)
# s.removeprefix(prefix)
# s.removesuffix(suffix)
# s.upper(), s.lower(), s.title(), s.casefold(), s.swapcase(), s.capitalise
```

```
# s.isupper(), s.islower(), s.istitle()
# s.isalpha(), s.isalnum(), s.isdigit()
```

Membership operator : in & not in

In [2]:
```
s = "hello"
```

In [4]:
```
print("o" in s)
```

```
True
```

In [6]:
```
print("o" not in s)
```

```
False
```

In [8]:
```
li = [10, 8+9j, "hi", 5.3]
```

In [10]:
```
print(10 in li)
```

```
True
```

In [12]:
```
print("o" not in li)
```

```
True
```

Problem-1 : Check if the password and confirm password is same or not.

In case of only case's mistake, show the error message.

In [15]:
```
pass1 = input("Enter your password:")
pass2 = input("Confirm your password:")
if pass1 == pass2:
    print("Passwords matched.")
elif pass1.lower() == pass2.lower():
    print("Check the case of your passwords.")
else:
    print("Passwords mismatched.")
```

```
Check the case of your passwords.
```

Problem-2 : Rearrange the given string. First lowercase then uppercase alphabets.

input : EHlsarwiwhtwMV

output : lsarwiwhtwEHMV

In [18]:
```
s = "EHlsarwiwhtwMV"
lw = ""
up = ""
for i in s:
    if i.islower():
        lw += i
    if i.isupper():
        up += i
```

```
ans = lw + up
print(ans)
```

```
lsarwiwhtwEHMV
```

Problem-3 : Remove punctuation

input : my_python@gmail.com

output : mypythongmailcom

In [21]:
```
s = "my_python@gmail.com"
pun = "_@."
ans = ""
for i in s:
    if i not in pun:
        ans += i
print(ans)
```

```
mypythongmailcom
```

Problem 4: Email ID Processing

Input: email = " Student.Name@GMAIL.com "

1. Remove extra spaces from both ends
2. Find the position of @ and the last .
3. Extract the username and domain name
4. Convert the domain name to lower case

expected output:

- Username : Student.Name
- Domain : gmail.com

In [24]:
```
s = "  Student.Name@GMAIL.com  "
s = s.strip()
pos1 = s.find("@")
pos2 = s.rfind(".")
print(f"Username : {s[:pos1]}")
print(f"Domain Name: {s[pos1+1:].lower()}")
```

```
Username : Student.Name
Domain Name: gmail.com
```

Problem 5: Word Occurrence and Replacement

Input: sentence = "python is easy and python is powerful"

1. Count how many times the word "python" appears
2. Find the first and last occurrence of "python"
3. Replace "python" with "Python"

Expected Output:

- Count : 2
- First occurrence : 0
- Last occurrence : 19
- Updated sentence : Python is easy and Python is powerful

In [33]:
```
s = "python is easy and python is powerful"
print(f"Count : {s.count("python")}")
print(f"First occurance : {s.find("python")}")
print(f"Last occurance : {s.rfind("python")}")
print(f"Updated sentence : {s.replace("python","Python")}")
```

```
Count : 2
First occurance : 0
Last occurance : 19
Updated sentence : Python is easy and Python is powerful
```

Problem 6: Student Name Formatting

Input: name = " sneha sharma "

1. Remove leading and trailing spaces
2. Convert the name to title case
3. Display the name in a width of 30 characters
   - Left aligned
   - Right aligned
   - Center aligned

In [38]:
```
name = "  sneha sharma  "
name = name.strip()
name = name.title()
print(f"{name.ljust(30)}")
print(f"{name.rjust(30)}")
print(f"{name.center(30)}")
```

```
Sneha Sharma
                  Sneha Sharma
        Sneha Sharma
```

Problem 7: Roll Number Standardization

Input: roll = "345"

1. Convert the roll number into a 6-digit format by adding leading zeros.
2. Check whether the roll number contains only digits.

Expected Output:

- Standard Roll Number : 000345
- Valid Roll Number : True

In [42]:
```
roll = "345"
roll = roll.zfill(6)
print(f"Standard Roll Number : {roll}")
print(f"Valid Roll Number :{roll.isdigit()}")
```

```
Standard Roll Number : 000345
Valid Roll Number :True
```

Problem 8: CSV Marks Processing

Input: marks = "78, 85, 90, 66, 72"

1. Split the string into individual marks
2. Remove extra spaces from each mark
3. Join the marks using | as a separator

Expected Output: 78|85|90|66|72

In [44]:
```
marks = "78, 85, 90, 66, 72"
li = marks.split(", ")
marks = "|".join(li)
print(marks)
```

```
78|85|90|66|72
```